the best choice depends on your requirements and context: if you have a dynamic set and expect many calls to delete, you'd better use pruning; if, instead, the ration insertion/removal is low, or you expect to have to add back deleted strings frequently or shortly after, then you are better off with the faster (though messier) removal shown in listing 6.7.

6.2.5 Longest Prefix

With remove, we completed our overview of the classic operations on containers; as we mentioned, though, tries can also provide two new kind of operations, which are the most exciting part of this data structure.

In this section we'll focus on the method that returns the longest prefix of the searched string; given an input string `s`, we traverse the trie following the path corresponding to characters in `s` (as long as we can), and return the longest key we found.

Sometimes, even if a key wasn't stored in our trie, we are just interested in getting its longest prefix: we'll see an example in the applications section.

The search for the longest prefix is almost entirely the same as the `search` method: the only difference is that, in a recursive implementation, when we backtrack we need to check if we have already found a key and, if we haven't and current node is a key node, we'll have to return current node's key. This also means that we have to return a string, not just true or false, and at each call keep track of the path